

Success & Failure & Fun

Why I love property testing

Deepak Cherian, Ph.D



Deepak Cherian

FORWARD ENGINEER
OCEANOGRAPHER
OSS MAINTAINER
FORMERLY AT NCAR, CGD OCEAN SECTION



Building cloud-native data
infrastructure at Earthmover PBC



What is property testing?

1. Write test case generators that sample the input space.
 - a. A property testing library usually provides helpers for these.
 - b. The helpers are “smart” : check NaN, Inf, INT_MAX, INT_MIN etc.
2. For each test case, assert that a property holds for the output of the tested function.
3. When a failing case is detected, a good property testing library will *shrink* the failing case to a much *simpler* case (defined somehow).

My take:

Unit test: I generate complexity → test specific examples

I am the “oracle” → I write out the correct answer to test against

Property test: I describe how to generate complexity → test abstract properties are satisfied

Sidestep the “oracle” problem

Over the past 18 months, I have built property testing suites for these open-source projects



Zarr

Chunked,
compressed
N-D Arrays

Data format

Stores data in any
key/value store : {"a": 0}

*Cannot lose or
misrepresent data*



Icechunk

Transactional Storage
Engine for Zarr

Emphasis on safety &
correctness over vanilla
Zarr

*Cannot lose or
misrepresent data*

xpublish-tiles

From Zarr to
web map

Given any dataset,
render a region of the
globe as an image.

*Must faithfully
represent data*

flox

Grouped aggregations
for arrays

Computation executed
in many ways (serial
,parallel, compiled)

*Supports broad range
of data types &
operations*

Why?

1. The stakes are high — data formats cannot lose data.
2. We want to move fast, with confidence.
3. I am on the hook if this breaks.
4. The input space is large. Covering it with unit tests is hard!
5. This is fun! (more fun than unit tests ;) and elegant! And a tool for thought!

Agenda

- 01 Where I have used property testing?
- 02 Examples of properties tested
- 03 Hands-on exercises!
- 04 Beware!
- 05 Lessons learned

A flavor of what property tests test

Choosing properties

Sources

- [John Hughes - How to specify it!](#)
 - “DO NOT THINK, let QuickCheck [hypothesis] do your thinking for you.”
- [Sean Wlaschin series on property testing](#)
- [Hillel Wayne on choosing properties](#)

The screenshot shows a presentation slide titled "Property Based Testing" from the Lambda Days conference (13-14 February 2020, Krakow, Poland). The slide features three bullet points with corresponding icons: a yellow smiley face for "Random generation of lots of test cases", a jagged green saw blade for "Shrinking results in minimal counterexamples—easy to debug", and a blue code block for "Replicating code in the tests is tempting, but expensive, and low value". A small video inset in the bottom right corner shows a man, John Hughes, gesturing while speaking. The slide footer includes the text "Keynote: How to specify it! A guide to writing properties of pure functions" and "John Hughes" with a website link.

Property Based Testing

- Random generation of lots of test cases
- Shrinking results in minimal counterexamples—easy to debug
- Replicating code in the tests is tempting, but expensive, and low value

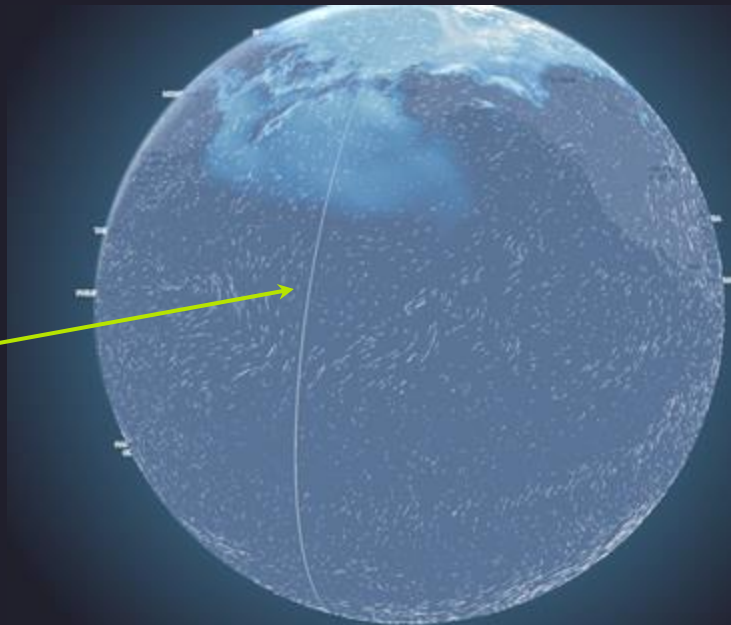
Keynote: How to specify it! A guide to writing properties of pure functions
John Hughes
www.jhmonaghan.org

1. Validity testing or invariants: xpublish-tiles

Property: For a dataset with global coverage, no tile should have a transparent pixel.

- Test over
 - Latitude: increasing or decreasing;
 - Longitude: $-180^\circ \rightarrow 180^\circ$, $0^\circ \rightarrow 360^\circ$
- Arbitrary data shapes (or grid spacings)
- Arbitrary zoom levels

This should
never happen



1. Validity testing or invariants: Zarr



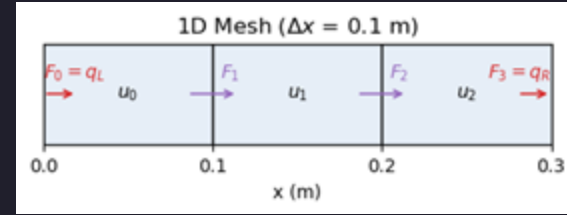
Property: we read what we write

- Generate an arbitrary in-memory array
 - Arbitrary data types (ints, floats, chars, datetimes etc.)
 - Arbitrary shapes
 - Arbitrary values
- Write to disk and read back
 - Over arbitrary store types (in-memory, object store, file storage, etc.)
 - Arbitrary folders
 - Arbitrary compression
 - Arbitrary “chunking”
 - And more...
- Must be identical to what was written (simple property)

1. Validity testing or “invariants”: heat equation

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}$$

With prescribed
boundary fluxes



1. Conservation of heat
2. Symmetry preservation: symmetric initial condition + symmetric boundary conditions == symmetric solution.
3. Monotonicity: non-decreasing initial states remain non-decreasing

We will look at how to implement these later in this session.

2. What is the postcondition?

“Construct inputs whose output is easy to predict.”

Property: Aggregating an array A after grouping by an array B of constant values is identical to aggregating the array A.

Example:

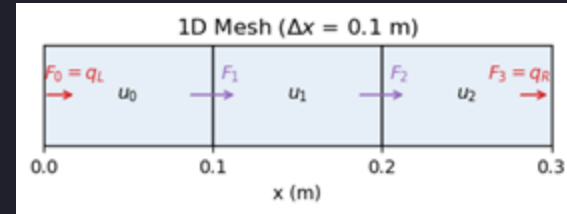
1. Consider A: array of arbitrary data type & shape e.g. [0, 1, 2];
2. B: array to group “by” of arbitrary data type but constant values e.g. [1, 1, 1], ['a', 'a', 'a']
3. `A.groupby(B).mean()` is identical to `A.mean()`
 - tests handling of inf, Nan
 - tests datatype casting rules etc.

2. What is the postcondition?

“Construct inputs whose output is easy to predict.”

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}$$

With prescribed
boundary fluxes



- u = arbitrary constant + 0 fluxes; $\Rightarrow u$ = arbitrary constant
- u = arbitrary linear function + 0 fluxes; $\Rightarrow u$ = arbitrary initial function
- u = constant + both boundary fluxes heat (or cool) $\Rightarrow du/dt > 0$ (< 0)

3. Metamorphic tests:

“related calls yield related results”

No oracle needed!

Spotify example ([doi:10.1109/TSE.2017.2764464](https://doi.org/10.1109/TSE.2017.2764464)):

Property: Search for “word1 word2” must have fewer results than the search for “word1”.

Flox example:

Property: forward filling and backward filling are inverse operations.

```
Consider array A = [1, 2, np.nan, 3]
                ffill(A) == reversed(bfill(reversed(A))) == [1, 2,
2, 3]
                bfill(A) == reversed(ffill(reversed(A))) == [1, 2, 3, 3]
```

3. Metamorphic tests: xpublish-tiles

“Different paths; same destination”

Property: The same data on a *uniformly spaced* grid should render as an equivalent tile regardless of how underlying coordinates are specified.

11	12
13	14

GeoTransform: [-0.5, 1, 0, -0.5, 0, 1]

Shape: 2 x 2

(a) Affine transform + shape

2	11	12
1	13	14

1	2
---	---

x

(b) Rectilinear mesh
1D x, y coordinates

2	2	11	12
1	1	13	14

1	2
1	2

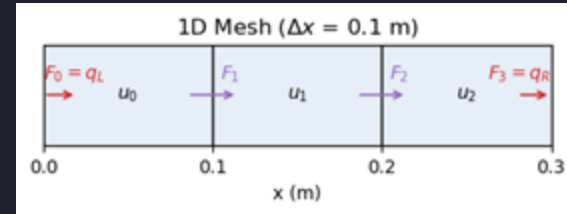
x

(c) Curvilinear mesh
2D x, y coordinates

3. Metamorphic tests: heat equation

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}$$

With prescribed
boundary fluxes



- Symmetry about the center
- Solution is unchanged upon shifting x coordinate; generalizes to dimensional consistency & non-dimensionalization
- Add an arbitrary constant to the initial condition; solution must be unchanged
- Solution with time step Δt and spacing Δx should be *approx* equal to that with $\Delta t/N$ (assuming stability condition etc...). Similar for $N * \Delta x$ (given Δt)

4. Model-based properties

Test against a simpler model (“implement a simple oracle”)

This only works if the model is simpler to implement.

Example:

- Indexing an array is identical to indexing that array after writing to disk in Zarr format
`numpy_array[0, slice(10), 3] == zarr_array[0, slice(10), 3]`

The heat equation could be a model for a different system (e.g. ocean model vertical mixing scheme).

Notebook

Explore the hypothesis library in a notebook.

- Hypothesis is a great (opinionated) property testing library!
- Open-source!
- Integrated with pytest
- Used by an estimated 5% of python developers
- Very UX focused
- Docs are great! <https://hypothesis.readthedocs.io/en/latest/>
- Releases every day or two!
- Easy integration with CI.



See `test_metamorphic.py`

Metamorphic testing

A variant of model-based property testing

(maybe) stateful testing

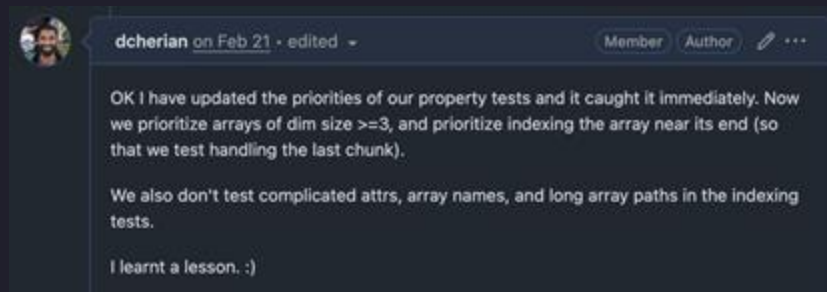
Property testing can lead to overconfidence

A failure

Lesson learned: examine your strategies!

1. I made a complex change to improve performance.
Unit tests passed, thought property test coverage was good.
2. Then a user reports data loss!
 - a. Zarr cuts an array in to “chunks” of uniform size; the last chunk can be smaller e.g. [2, 2, 1]
 - b. I introduced a bug when overwriting the last chunk when it is smaller.
3. Why? We spent all our effort testing parts of parameter space that weren't relevant to most users or irrelevant to the read/write operation being tested
 - i. arrays of shape (0,) and (1,); complicated unicode attributes etc.And almost no effort testing the most common “complex” case (a smaller last chunk).

Pull Request



Lessons learned

01.

Property tests **don't replace**
other testing strategies.

They are meant to help you increase coverage
(and think through the problem).

02.

Property tests **pseudo-randomly** sample the input search space.

This means it is upon you to narrow the search space to “interesting” regions. Hypothesis can help.

Minimize the search space along dimensions that are unimportant for the property you are testing.

A successful test doesn’t prove anything! Use CI.

Zarr nightly CI

✓ Slow Hypothesis CI Slow Hypothesis CI #5755: Scheduled	main	Nov 4, 5:06 PM MST 13m 22s	*** 🗑
✓ Slow Hypothesis CI Slow Hypothesis CI #5750: Scheduled	main	Nov 3, 5:06 PM MST 13m 56s	*** 🗑
✓ Slow Hypothesis CI Slow Hypothesis CI #5735: Scheduled	main	Nov 2, 5:06 PM MST 13m 30s	*** 🗑
✓ Slow Hypothesis CI Slow Hypothesis CI #5727: Scheduled	main	Nov 1, 6:06 PM MDT 12m 51s	*** 🗑
✓ Slow Hypothesis CI Slow Hypothesis CI #5723: Scheduled	main	Oct 31, 6:07 PM MDT 13m 36s	*** 🗑
✓ Slow Hypothesis CI Slow Hypothesis CI #5718: Scheduled	main	Oct 30, 6:06 PM MDT 11m 5s	*** 🗑

03.

Simple properties are **really** effective
and **very** elegant.

Explore complexity in input generation
(explore the input space).

Think of this as a thinking tool.

04.

Property testing suites are effectively an **empirical specification** of your program's behaviour.

If you don't like it; time to fix it!

05.

Your unit test is most likely a property test in disguise.

Generalize it!

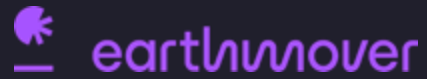
rewrite `@parametrize` to a combination of `@given` & `@example`

06.

When you find a bug, construct a property test to catch it!

But beware, property tests can lead to overconfidence.

They don't replace other testing strategies!



Thank you!

`deepak@cherian.net`